项目代码全解析（逐文件）

> 目的：让你能**逐段讲清自己项目里的每一块代码**——它干什么、怎么实现、为什么这么设计、哪里容易错。 覆盖：主控 F407、传感器节点 F103、电机节点 F103、Bootloader F407、上位机 Python。 配合《项目深度自测题库》使用：题库考你，本文给你标准答案的来源。

---

0. 总览：代码地图

每个节点都是同一套分层结构：

```
应用层    app_arch.c / app_can.c   任务、协议、命令应答
驱动层    app_motor.c / app_mpu6050.c / bl_flash.c ...
外设层    can.c / usart.c / gpio.c  HAL 初始化（CubeMX 生成）
```

工程	关键文件	职责
master_node (F407)	`app_arch.c`	系统架构：任务/队列/事件组/互斥量/故障监控
	`app_can.c`	心跳、命令、过滤器、接收回调
sensor_node (F103)	`app_can.c`	收 0x101、发 0x102/0x103
	`app_mpu6050.c`	软件 I2C 读六轴
motor_node (F103)	`app_motor.c`	PWM、编码器、PID
	`app_can.c`	收 0x201、闭环、发 0x202
bootloader (F407)	`bl_protocol.c` / `bl_flash.c` / `bl_crc.c`	IAP 升级
tools	`upgrade_tool.py` / `proto_smoke.py`	PC 端升级工具

数据流：

```
Node1 温度0x102 / 加速度0x103 ─┐
                               ├─→ F407 CanRxTask ─→ 状态表 ─→ ControlTask ─→ MotorTask
Node2 速度0x202 ──────────────┘
F407 CanTxTask ──→ 0x100 心跳(1s) / 0x101、0x201 命令(每5s交替)
```

> 注意：`master_node/archive/` 里的 `app_queue.c`、`app_event.c` 等是 Day7-13 的**FreeRTOS 练习归档**，不参与当前系统，阅读时跳过。

---

1. 主控 F407（master_node）

1.1 `main.c` — 启动流程

做什么：上电后初始化时钟和外设，创建 RTOS 对象，启动调度器。

关键顺序：

```
SCB->VTOR = 0x08010000U;   /* 向量表偏移到 APP 区（IAP 布局） */
HAL_Init();
SystemClock_Config();      /* HSE 8M → PLL → 168MHz */
MX_GPIO_Init(); MX_USART1_UART_Init(); MX_TIM3_Init(); MX_CAN1_Init();
printf("F407 App v1.0 Start!\r\n");
HAL_TIM_Base_Start_IT(&htim3);
MX_FREERTOS_Init();
osKernelStart();           /* 启动调度器，不再返回 */
```

要点 / 为什么：

- `SCB->VTOR = 0x08010000`：仓库里主控副本是按 IAP 布局的（APP 在 `0x08010000`）。中断向量表必须跟着搬，否则中断会按 `0x08000000` 查表跑飞。
- **时钟**：HSE 8MHz，`PLLM=8, PLLN=336, PLLP=2` → 168MHz；`APB1 = /4 = 42MHz`（CAN1 挂在 APB1，所以 CAN 时钟按 42MHz 算）。
- `HAL_TIM_PeriodElapsedCallback`：TIM3 溢出中断里 `xSemaphoreGiveFromISR` 给一个二值信号量——这是 ISR 通知任务的典型用法。

1.2 `freertos.c` — RTOS 初始化

做什么：创建互斥量与任务，挂上内核钩子。

- `uartMutexHandle = osMutexCreate(...)`：保护串口打印。
- `xBinarySemaphore = xSemaphoreCreateBinary()`：给 TIM3 中断用。
- `defaultTask`：1s 翻转 LED（心跳灯）。
- `archTask`（优先级 Low）：入口 `ArchDemoTask`，**真正创建系统任务**。
- 钩子：
  - `vApplicationMallocFailedHook`：堆分配失败打印（用临界区，不是互斥量——钩子可能在调度器启动前被调用）。

- `vApplicationStackOverflowHook`：栈溢出打印**并复位**（栈已坏，状态不可信）。
- `vApplicationGetIdleTaskMemory` / `vApplicationGetTimerTaskMemory`：空闲/定时器任务用**静态内存**。

1.3 `app_arch.c` — 系统架构（全项目最核心）

做什么：定义系统状态、内核对象、六个任务，以及 ISR 投递入口。

系统状态（由 `xSysMutex` 保护）：

```
typedef struct {
    uint32_t uptime_tick;
    uint8_t  node_online[2];
    int32_t  node_temp[2];
    uint16_t adc, speed;
    uint32_t last_rx_tick[2];   /* 每个节点最近收帧时刻 → 离线检测依据 */
    uint8_t  fault_temp, fault_offline;
} SysState_t;
```

内核对象：`xCanRxQueue`(8×CAN帧)、`xMotorQueue`(4×u16)、`xSysEvents`、`xSysMutex`、`xHeartbeatTimer`(1s)。

关键函数：

- **`ArchPostCanFrame()`**（ISR 安全）：把帧投递到队列 + 置事件位 + 触发调度。

  ```
  xQueueSendFromISR(xCanRxQueue, pFrame, &woken);
  xEventGroupSetBitsFromISR(xSysEvents, EVT_NEW_FRAME, &woken);
  portYIELD_FROM_ISR(woken);
  ```

- **`ArchPrint()`**：加 `uartMutex` 再 `vprintf`，防止多任务打印交错。
- **`HeartbeatCallback()`**：软件定时器回调（在定时器任务上下文），`uptime_tick++` 并置 `EVT_HEARTBEAT`；取互斥量用**非阻塞** `xSemaphoreTake(..., 0)`。

六个任务（优先级体现实时性）：

任务	优先级	职责
CanRxTask	4	收帧、更新状态表（`last_rx_tick`、`node_online`、温度/速度）
ControlTask	5	等新帧/心跳 → 温度越限告警 → 算目标速度 → 发 `xMotorQueue`
MotorTask	3	收目标速度并打印（Day15 起转发给电机节点）
FaultMonitorTask	4	每心跳检查 `now-last_rx_tick > 5000ms` → 判离线
CanTxTask	2	1s 发心跳 0x100；每 5s 交替发 0x101/0x201

任务	优先级	职责
DebugTask	1	2s 打印状态与堆水位

`ArchDemoTask`：创建所有对象 → 创建任务 → `CanStart()` → 自删。这是“主控启动”的真正入口。

为什么这么设计（面试点）：

- **按职责拆任务**（接收/控制/故障/发送/调试），单一职责、低耦合。
- **优先级按实时性排**：控制/接收/故障 > 电机 > 发送 > 调试。
- **同步选型**：传数据用**队列**、告警用**事件组**、保护共享用**互斥量**、定时用**软件定时器**。
- **中断最小化**：ISR 只入队+置位，解析和业务在任务里。

1.4 `app_can.c` — CAN 应用层

`CanStart()`：过滤器 Bank0 收 `0x102`、Bank1 收 `0x202`（掩码 `0x7FF`，`ID<<5` 对齐），启动外设并开启 RX0/错误/BusOff/LEC 中断。

> 这就是为什么主控**收不到 0x103**——过滤器里没配它。

`HAL_CAN_ErrorCallback()`：只把 `hcan->State = HAL_CAN_STATE_READY`，**不在 ISR 里 printf**。F1/F4 HAL 错误中断后状态机会锁死，不恢复就再也发不出去。

`CanSendHeartbeat(seq)`：发 0x100，DLC=8，负载 = `seq` 小端 + `0x55AA`；发完**直接读 ESR** 打印 `LEC/TEC/REC`（诊断不依赖中断）。

`CanSendCommand(node)`：

- `node=0` → 0x101，DLC=1，`data[0]=0x01`（请求 Node1 上报温度）
- `node=1` → 0x201，DLC=3，`data[0]=0x01` + 目标速度 500/1000 **小端**

`HAL_CAN_RxFifo0MsgPendingCallback()`：取帧 → 记 tick → **ID 映射节点号**（`0x202→1`，否则 `0`）→ `ArchPostCanFrame()`。

1.5 `can.c` — CAN 外设初始化（F407）

```
hcan1.Init.Prescaler = 6;
hcan1.Init.TimeSeg1 = CAN_BS1_11TQ;
hcan1.Init.TimeSeg2 = CAN_BS2_2TQ;    /* 42M/6/14 = 500k，采样点 85.7% */
hcan1.Init.AutoBusOff = DISABLE;      /* F407 这边没开自动恢复 */
```

MSP：**PA11/PA12 配** `AF_PP + AF9_CAN1`（F4 有复用功能选择寄存器）——这正和 F103 相反，是面试高频对比点。

---

2. 传感器节点 F103（sensor_node）

2.1 `main.c`

F103 时钟：HSE ×9 = **72MHz**，`APB1 = /2 = 36MHz`（CAN1 挂 APB1）。没有 VTOR 设置（F103 不做 IAP 布局）。初始化 GPIO/USART1/CAN1 后启动调度器。

2.2 `freertos.c`

- `NodeCanInit()`：**在调度器启动前**创建 CAN 接收队列。
- `uartMutexHandle`：串口互斥。
- 任务：`defaultTask`（LED）、`canRxTask`（AboveNormal）、`canTxTask`（Normal）。
- 同样的三个钩子（malloc 失败、栈溢出复位、idle/timer 静态内存）。

2.3 `app_can.c` — 收 0x101、发 0x102/0x103

`CanStart()`：

```
/* 重新声明 PA11=INPUT+PULLUP，PA12=AF_PP —— F103 关键修复 */
CAN1->MCR &= ~CAN_MCR_DBF;          /* 清调试冻结位 */
/* 过滤器：Mask=0x7FF，Filter=0x101 → 只收命令帧 */
```

为什么：F1 没有 AF 选择寄存器，CAN_RX 必须配**输入模式**；配成 `AF_PP` 会退不出初始化（INAK 恒 1）。

`CanSendSensorData(temp)`：0x102，DLC=2，`data[0]=temp, data[1]=0`（温度 <256）。

`HAL_CAN_RxFifo0MsgPendingCallback()`：取帧 → `xQueueSendFromISR`。

`CanRxTask`：收 0x101 且 `data[0]==0x01` → **立即回 0x102**（请求/应答闭环）。

`CanTxTask`：

- 先 `CanStart()` + `MPU_Init()`；
- 每 5s 打印一次 CAN 状态快照（State/LEC/TEC/REC/PCLK1/SYSCLK）；
- 每 1s 发 0x102 温度（30→60 循环递增）；
- 读 MPU6050，串口打印，并组 **0x103** 帧（DLC=6，`ax/ay/az` 各 2 字节小端）广播。

2.4 `app_mpu6050.c` — 软件 I2C 驱动（很值得讲）

为什么用软件 I2C：F1 的硬件 I2C 有已知兼容性问题，软件 I2C 更稳。

引脚：PB6=SCL、PB7=SDA，配 **开漏输出 + 上拉**（`GPIO_MODE_OUTPUT_OD`）——开漏才能实现“线与”，配合模块上拉。

核心时序（用位操作和 GPIO 读写手写）：

- `mpu_i2c_start()`：SCL 高时 SDA 拉低
- `mpu_i2c_stop()`：SCL 高时 SDA 拉高
- `mpu_i2c_wait_ack()`：第 9 个时钟读 SDA，低=ACK
- `mpu_i2c_write_byte()` / `mpu_i2c_read_byte(ack)`：MSB 在前
- `mpu_write_reg()` / `mpu_read_bytes()`：读时用**重复起始**切换读方向

`MPU_Init()`：配开漏 GPIO → 写 `PWR_MGMT_1=0`（退出睡眠）、`SMPLRT_DIV=7`（125Hz）、`CONFIG`、`GYRO_CONFIG=0`（±250°/s）、`ACCEL_CONFIG=0`（±2g）→ 读 `WHO_AM_I` 校验 == `0x68`。

`MPU_ReadAll()`：从 `0x3B` 起**突发读 14 字节**（accel 6 + temp 2 + gyro 6），按**大端**拼成 `int16`。

> 注意：MPU6050 寄存器数据是**大端**（`buf[0]<<8 | buf[1]`），和 CAN 协议的小端不一样——面试可以拿这个讲“端序看待对象”。

2.5 `can.c`（F103）

```
hcan1.Init.Prescaler = 4;
hcan1.Init.TimeSeg1 = CAN_BS1_15TQ;
hcan1.Init.TimeSeg2 = CAN_BS2_2TQ;     /* 36M/4/18 = 500k，采样点 88.9% */
hcan1.Init.AutoBusOff = ENABLE;        /* F103 这边开了自动恢复 */
```

MSP 注释明确提醒：**PA11 必须配** `GPIO_MODE_INPUT`，且**不要调用** `__HAL_AFIO_REMAP_CAN1_1()`（会把 CAN 挪到 PB8/PB9）。

---

3. 电机节点 F103（motor_node）

3.1 `main.c`

同 F103 时钟（72MHz），初始化 GPIO/USART1/CAN1，打印 `F103 Node2 Motor Start!`。

3.2 `freertos.c`

与传感器节点结构一致：`NodeCanInit()` + `uartMutex` + `defaultTask` / `canRxTask` / `canTxTask` + 三个钩子。

3.3 `app_motor.c` — PWM / 编码器 / PID

`MotorInit()`：

- TIM2 CH1(PA0) PWM：`PSC=35 → 2MHz`，`ARR=999 → 2kHz`；

- PA1 方向脚，默认拉低（双低=停转，避免上电即转）；
- TIM3 编码器模式 `TIM_ENCODERMODE_TI12`（A/B **4 倍频**），PA6/PA7 **上拉**（抗噪声）；
- 本地静态 `htim2/htim3`，不依赖 CubeMX 生成。

`MotorSetDuty(duty)`：钳到 `MOTOR_DUTY_LIMIT=800`，避免 `CCR>ARR` 恒高导致 L298N 刹车。

`MotorSetDir(dir)`：`dir=1` → PA1 低（正向）。注释记录了 L298N 极性坑：IN2 恒高时占空比越大越接近刹车。

`MotorPidCompute(target_rpm, rpm_fb)`（核心）：

```
int32_t e  = target - rpm_fb;
int32_t dout = PID_KP*(e - pid_err1)
             + ((|e|>PID_INT_SEP_ERR)?0:PID_KI)*e
             + PID_KD*(e - 2*pid_err1 + pid_err2);
pid_out += dout;  /* 增量式累加 */
if (pid_out > 800) pid_out = 800;
if (pid_out < 0)   pid_out = 0;
```

增量式 + **积分分离**（阈值 30）+ **输出限幅**。

3.4 `app_can.c` — 收 0x201、闭环、发 0x202

- `CanStart()`：同 F103 修复（PA11 输入 + 清 DBF），过滤器只收 `0x201`。
- `CanSendMotorData(speed)`：0x202，DLC=2，小端。
- `CanRxTask`：收 0x201 且 `data[0]==0x01` → 解析目标速度 → `MotorSetDir(1)` → 回 0x202。
- `CanTxTask`（20ms 周期，闭环主体）：

```
enc = MotorGetCount(); delta = enc - last_enc;  /* 取绝对值防回绕 */
rpm = delta*60000/(dt*ENC_COUNTS_PER_REV);       /* 实际 dt 换算 */
rpm_sm = (rpm_sm*3 + rpm)/4;                     /* EMA 平滑 */
target_rpm = g_speed/2;
duty = MotorPidCompute(target_rpm, rpm_sm);
if (g_speed==0) duty = 0;
MotorSetDuty(duty);
/* 每 1s 累加差值算 RPM 发 0x202 */
vTaskDelay(pdMS_TO_TICKS(20));
```

要点：编码器测速用**实际 dt**（不假设固定 20ms）；`g_speed/2` 是协议值到 RPM 的近似映射；反馈先 EMA 再进 PID。

---

4. Bootloader（F407）

4.1 `main.c` — 启动判断

```
printf("BOOT v0.1\r\n");
/* 1s 握手：u=升级，j=强制跳转，超时=默认 */
if (got && ch=='u')      BlProtocolRun();   /* 不返回 */
else if (got && ch=='j') BlJumpToApp();     /* 调试用，不校验 */
else {
    if (BlAppValid()) BlJumpToApp();
    printf("NO APP\r\n");
    while(1){ HAL_Delay(1000); }
}
```

设计意图：给 1s 窗口，PC 发 `u` 就进升级；否则校验 APP 有效性，有效就跳转，无效停在 `NO APP` 等升级。

4.2 `bl_flash.c` — 擦写 / 有效性 / 跳转

- `AddrToSector()`：地址→扇区号（Sector 0-3 是 16KB，Sector 4 是 64KB，Sector 5-11 是 128KB）。
- `BlFlashEraseRange()`：按扇区擦除 `[addr, addr+len)` 覆盖的范围（**擦除只能整扇区**）。
- `BlFlashWriteRange()`：攒成 **64 位双字**再 `HAL_FLASH_Program`，尾块补 `0xFF`。
- `BlFlashWriteMagic()`：在 `APP_START+0x100` 写 `0xA5A5A5A5`。
- `BlAppValid()`：栈指针在 `0x20000000~0x20020000`（128KB SRAM）**且** magic 正确。
- `BlJumpToApp()`：关中断 → 读 APP 首字设为 MSP → 取 `APP_START+4` 的 Reset_Handler 用**函数指针**跳转。

关键常量：`APP_START=0x08010000`、`APP_LEN_MAX=0xF0000`(960KB)、`APP_MAGIC=0xA5A5A5A5`。

4.3 `bl_protocol.c` — 升级协议状态机

帧状态机（`stage` 变量）：

1. `stage0/1`：等 `A5 5A` 帧头；
2. `stage2`：收 `CMD(1)+ADDR(4)+LEN(2)` 共 7 字节，解析 `plen`，超 256 丢弃；
3. `stage3`：收 `DATA(plen)+CRC32(4)`，对 **DATA 段**算 CRC，比对：
   - 不符 → 回 `NACK(0x15)`；
   - 相符 → `BlDispatchFrame()`。

命令分发 `BlDispatchFrame()`（`hdr[0]=CMD, hdr[1..4]=ADDR, hdr[5..6]=LEN, hdr[7..]=DATA`）：

- `ERASE`：`BlFlashEraseRange(APP_START, LE32(&hdr[7]))`（DATA 是 4 字节镜像总长）
- `WRITE`：先校验 `addr` 落在 APP 区，再 `BlFlashWriteRange`（**防越界覆盖 Bootloader**）
- `FINISH`：`BlFinishImage(总长, CRC)` → 对 Flash 重算 CRC，比对通过才写 magic
- `REBOOT`：`NVIC_SystemReset()`
- 非 REBOOT 都回 `ACK(0x06)` / `NACK`

4.4 `bl_crc.c` — CRC32

逐位反射实现，多项式 `0xEDB88320`，初值/末值异或 `0xFFFFFFFF`，与 Python `zlib.crc32` 一致（向量 `0xCBF43926` 验证）。

4.5 `bl_uart.c`

- `UART_Send()`：阻塞发送（超时 100ms）。
- `UART_GetByte()`：`RXNE` 置位才调 `HAL_UART_Receive`，**非阻塞**读一字节。

---

5. 上位机工具（tools/）

5.1 `upgrade_tool.py`

- **`frame(cmd, addr, payload)`**：组帧 `A5 5A | CMD | ADDR(4 LE) | LEN(2 LE) | DATA | CRC32(4 LE, 仅 DATA)`。
- **`wait_ack()`**：轮询读 1 字节，`0x06`=ACK，`0x15`=NACK，超时返回 None。
- **`send()`**：发一帧等 ACK，超时最多重试 3 次。
- **`main()`**：读 bin → 开串口 → 等 `BOOT` → 发 `u` → `ERASE` → 每 256B `WRITE` → `FINISH`(总长+全图 CRC) → `REBOOT`。

5.2 `proto_smoke.py`

逻辑同上，但用 `b"\xAA"*4096` 假镜像做**冒烟测试**，在没有真实 APP 时先验证“擦-写-校验-重启”链路通不通。

---

6. 代码 ↔ 协议 ↔ 参数 速查

位置	关键内容
主控过滤器	Bank0=0x102，Bank1=0x202（不含 0x103）
传感器过滤器	0x101

位置	关键内容
电机过滤器	0x201
主控任务	CanRx(4)/Ctrl(5)/Motor(3)/Fault(4)/CanTx(2)/Debug(1)
主控周期	心跳 1s；命令每 5s 交替；离线超时 5s
传感器周期	0x102 每 1s；0x103 每 1s
电机周期	PID 20ms；0x202 每 1s
关键参数	PPR=1079；PID Kp=5/Ki=1/Kd=0，分离阈值 30，限幅 [0,800]
PWM	TIM2，2kHz，ARR=999
Bootloader	APP=0x08010000，960KB；magic=0xA5A5A5A5@+0x100
升级帧	A5 5A + CMD + ADDR(4LE) + LEN(2LE) + DATA + CRC32(4LE)

7. 从代码里能提炼的面试点

1. **任务/同步选型**：为什么 CAN 接收用队列、打印用互斥量、告警用事件组。
2. **中断最小化**：ISR 只入队+置位，解析在任务。
3. **F1 vs F4 的 CAN 引脚差异**：F1 必须输入模式，F4 用 AF_PP+AF9。
4. **过滤器掩码 + ID<<5**：为什么、怎么精确收帧。
5. **错误处理**：TEC/REC/LEC、ErrorCallback 恢复 READY、BusOff。
6. **软件 I2C**：为什么手写、开漏+上拉、起始/停止/ACK 时序。
7. **端序**：CAN 协议小端 vs MPU6050 寄存器大端。
8. **增量式 PID + 积分分离 + 限幅**：为什么、参数怎么来。
9. **Flash 扇区擦除**：只能整扇区、最大 128KB、先擦后写。
10. **IAP**：VTOR、MSP 设置、函数指针跳转、magic 有效性、CRC 全镜像校验。